



Agri Twin – Digital Twin Platform for Precision Farming and Crop Health Prediction

CI/CD Pipeline Design Exercise

Submitted in the partial fulfilment for the Course of
CSA1012 – Software Engineering

to the award of the degree of

BACHELOR OF TECHNOLOGY

Submitted by

Paleru Dharani Govardhan (192525280)

UNDER THE SUPERVISION OF

Dr. K. Anita Davamani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

July-2026

1. Project Overview and CI/CD Requirements

1.1 Industry Context

Agriculture is undergoing rapid digital transformation as farmers increasingly adopt smart technologies to improve productivity, sustainability, and resource efficiency. Climate change, unpredictable weather, soil degradation, pest infestations, and water scarcity continue to threaten crop yields worldwide. Precision agriculture leverages IoT sensors, satellite imagery, drones, and artificial intelligence to monitor field conditions and optimize farming practices. Digital Twin technology creates a virtual representation of agricultural fields by integrating real-time and historical farm data for continuous monitoring and simulation, enabling AI-powered predictive models to support early detection of crop stress, diseases, and nutrient deficiencies.

1.2 Problem Statement

Farmers often rely on manual field inspections and traditional practices that fail to identify crop health issues before they significantly affect productivity. Existing precision-farming solutions typically provide isolated monitoring features rather than an integrated virtual model of the farm for predictive analysis. Delayed detection of diseases, pests, irrigation problems, and nutrient deficiencies leads to reduced yields and higher operational costs, compounded by fragmented data from multiple sources. AgriTwin addresses this gap through an intelligent Digital Twin platform that continuously synchronizes real-time farm data, predicts crop health conditions, and recommends optimized farming practices, supporting proactive, AI-driven farm management.

1.3 Project Objectives

1. Develop a Digital Twin platform that creates a virtual representation of agricultural fields.
2. Collect real-time data from IoT sensors, drones, weather stations, and satellite imagery.
3. Predict crop health, diseases, and pest outbreaks using AI and machine learning models.
4. Monitor soil moisture, nutrient levels, irrigation status, and environmental conditions.
5. Recommend precision farming practices for irrigation, fertilization, and pest management.
6. Provide visualization dashboards for farm performance and crop growth monitoring.
7. Generate predictive reports to support data-driven agricultural decision-making.
8. Improve resource utilization while promoting sustainable and climate-smart farming.

1.4 Expected Outcomes

- Improved crop yield through early detection of crop health issues.
- Reduced losses caused by diseases, pests, and adverse environmental conditions.
- Efficient utilization of water, fertilizers, and agricultural resources.
- Enhanced decision-making through Digital Twin simulations and predictive analytics.
- Increased farm productivity and operational efficiency.
- Real-time monitoring of crop and environmental conditions.
- Better sustainability through optimized precision farming practices.
- A scalable smart agriculture platform supporting modern digital farming ecosystems.

1.5 CI/CD Requirements

AgriTwin integrates IoT data ingestion, a Digital Twin backend, an AI/ML prediction service, and user-facing dashboards. Each of these components changes independently and must remain reliable as new features are added. A CI/CD pipeline is required so that every change is automatically validated, packaged, and safely released without manual, error-prone steps. The table below connects specific AgriTwin characteristics to the corresponding CI/CD requirement.

AgriTwin Characteristic	CI/CD Requirement	Reason
IoT data integration	Integration testing	Validate data ingestion
Digital Twin	Backend testing	Ensure model synchronization
AI/ML prediction	ML validation	Validate prediction pipeline
Crop health prediction	Regression testing	Prevent model/API failures
Dashboard	Frontend/build testing	Ensure interface reliability
Real-time monitoring	Deployment health checks	Detect failures
Scalable platform	Docker/Kubernetes	Support scalability
Predictive reports	API/integration testing	Validate report generation

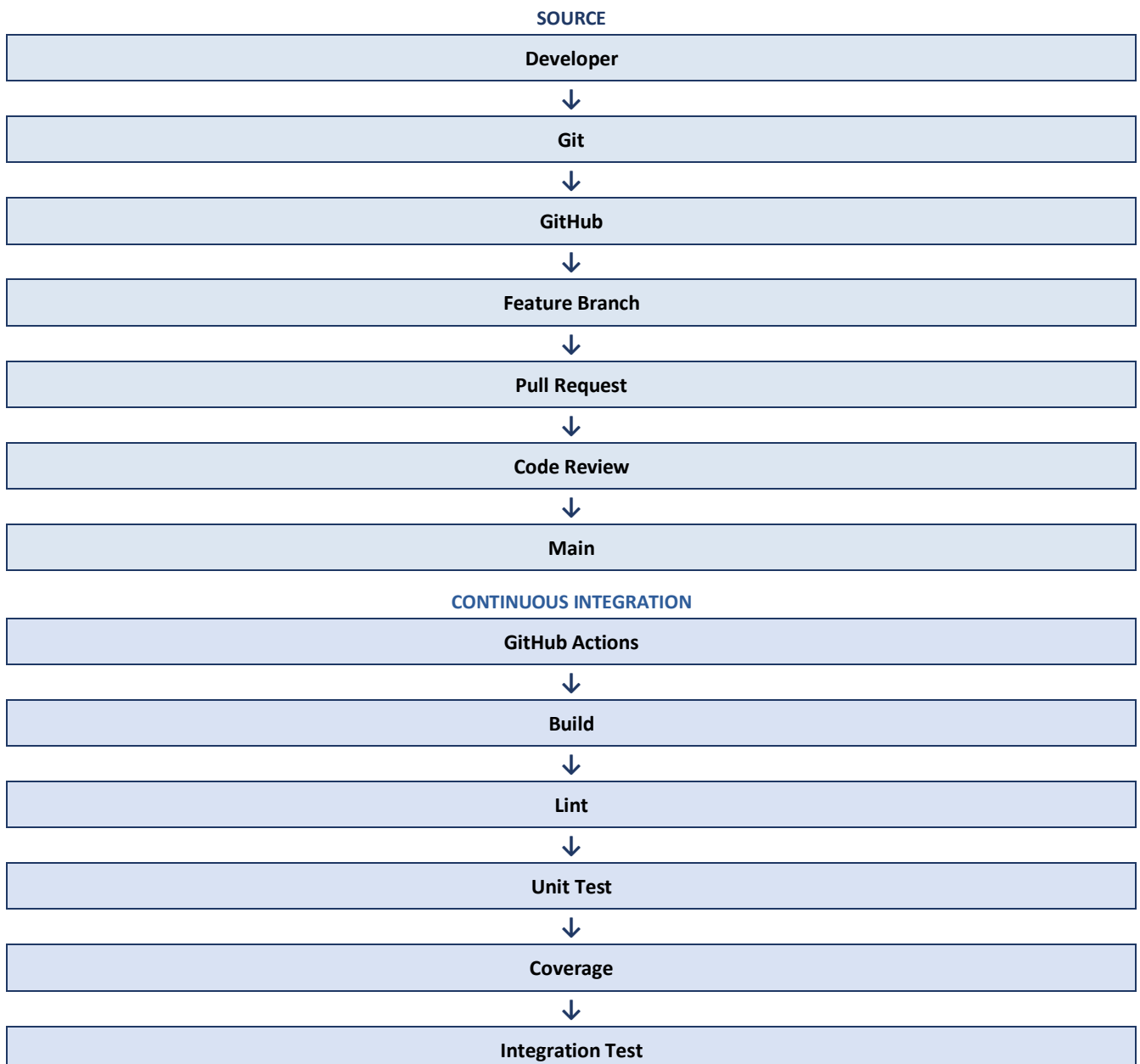
2. CI/CD Architecture

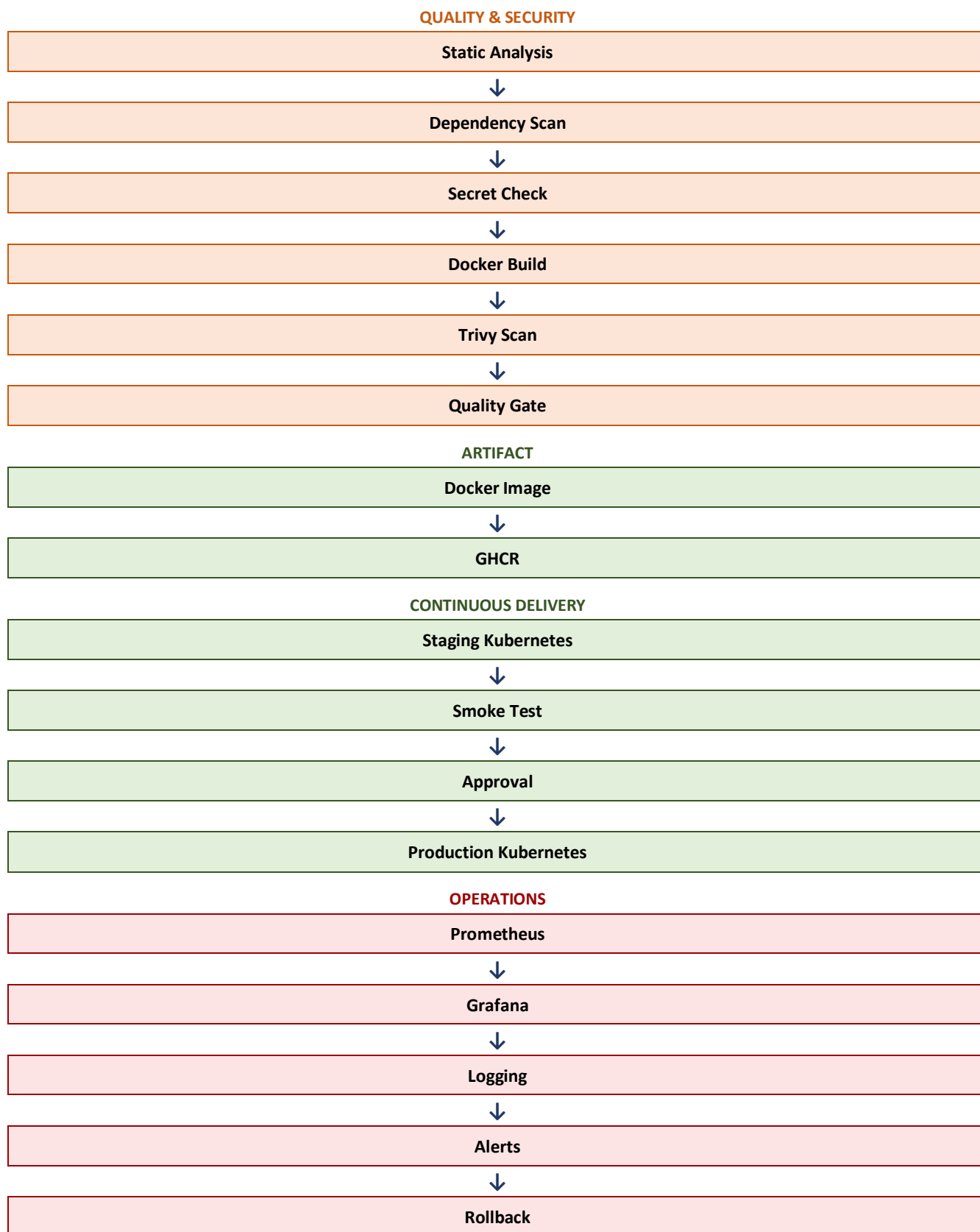
2.1 CI/CD Objectives

The AgriTwin pipeline is designed around the following core objectives:

- Continuous Integration – merge and validate code changes frequently through automated build and test execution.
- Continuous Testing – run unit, API, integration, and ML validation tests at every change.
- Continuous Delivery – keep the application in a consistently deployable state via staging.
- Continuous Deployment – automate promotion to production once quality gates are satisfied.
- Security – detect vulnerabilities in dependencies, containers, and secrets early.
- Quality Assurance – enforce linting, coverage, and static analysis before merge.
- Reliability – use health checks, monitoring, and rollback to protect production.
- Repeatability – produce identical, reproducible builds through containerization and IaC.

2.2 AgriTwin CI/CD Architecture

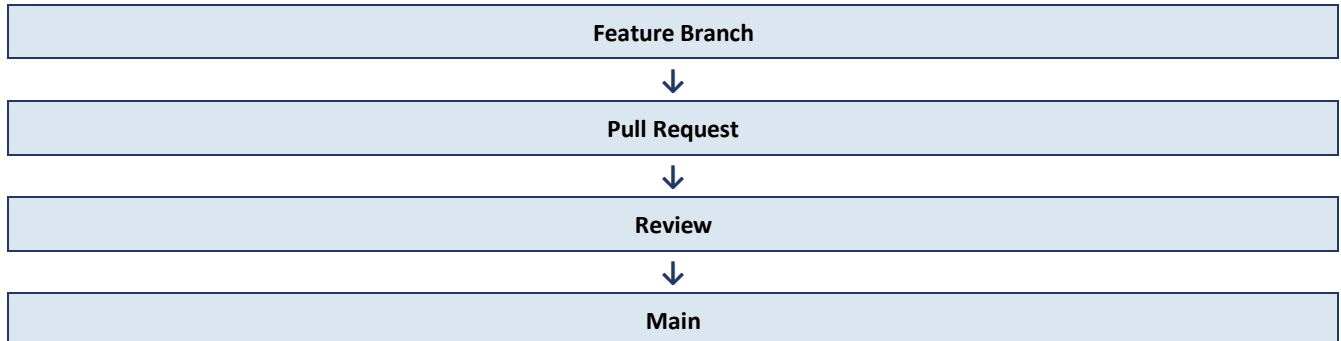


*Figure 2.1: AgriTwin CI/CD Architecture Overview*

3. Pipeline Workflow

3.1 Source Code Management

AgriTwin uses Git for version control and GitHub for collaborative development. All work happens on feature branches, merged into main only after a pull request and peer code review.



3.2 Pipeline Trigger

- pull_request – runs validation checks before merge.
- push (to main) – runs the full CI pipeline and staging deployment.
- release/tag – triggers the production deployment workflow.

3.3 Build

The build stage installs Python dependencies and validates that the Flask application and supporting services import and initialize correctly, catching configuration errors early.

3.4 Code Quality

Ruff or Flake8 performs linting and static analysis to enforce coding standards and catch common defects before tests run.

3.5 Automated Testing

- Unit tests – isolated logic (validation, processing, recommendations).
- API tests – request/response contracts for backend endpoints.
- Integration tests – cross-component data flow across the Digital Twin stack.

3.6 AI/ML Validation

The AI/ML component requires validation of input schema, preprocessing steps, model loading, prediction output, expected output format, and model/version compatibility. No accuracy figures or datasets are asserted in this design; actual metrics must be captured during implementation.

3.7 Security

Dependency scanning identifies vulnerable third-party packages; container scanning (Trivy) inspects the built Docker image for known CVEs before it is promoted.

3.8 Docker Packaging

A validated build is packaged into an immutable, versioned Docker image suitable for deployment to any environment.

3.9 Staging Deployment

The image is deployed to a staging Kubernetes namespace that mirrors production, where smoke and integration checks verify the release before promotion.

3.10 Production Deployment

Following manual or automated release approval, the image is deployed to production using a controlled rolling update, monitored by health checks.

Stage	Input	Action	Output	Gate
Checkout	Git commit	Fetch source	Workspace	Branch exists
Build	Source code	Install deps	Build artifact	Build succeeds
Lint	Source code	Static analysis	Lint report	No lint errors
Test	Build artifact	Run test suites	Test report	Tests pass
Scan	Deps/Image	Vulnerability scan	Scan report	No critical CVE
Package	Validated code	Docker build	Docker image	Build succeeds
Deploy Staging	Docker image	K8s apply	Running pods	Health check
Deploy Prod	Approved image	K8s rollout	Running pods	Health check

4. Tools and Technology Selection

The following stack was selected for its maturity, tight GitHub integration, and suitability for a Python/Flask, containerized, Kubernetes-based platform. Each tool is used only where it adds clear value; no unnecessary technologies are introduced.

Stage	Tool	Purpose	Justification
Source Control	Git / GitHub	Version control, collaboration	Industry standard; native GitHub Actions integration
CI/CD	GitHub Actions	Pipeline automation	Tightly coupled to the GitHub repository; no external CI server to maintain
Backend	Python / Flask	Application/API layer	Lightweight, well suited to data and ML-driven services
AI/ML	Python ML service	Crop health prediction	Consistent language with backend; easy to test in CI
Testing	Pytest	Unit/API/integration tests	De-facto Python testing framework with rich plugin ecosystem
Code Quality	Ruff / Flake8	Linting, static analysis	Fast, low-configuration Python linting
Coverage	Coverage.py	Test coverage measurement	Standard coverage tool for Pytest projects
Security	Trivy	Dependency & image scanning	Single tool covering both dependency and container vulnerabilities
Containerization	Docker	Packaging	Reproducible, portable runtime environments
Registry	GHCN	Image storage	Native GitHub integration; simplified authentication
Orchestration	Kubernetes	Deployment, scaling	Supports rolling updates, health checks, and scalability
Monitoring	Prometheus	Metrics collection	Standard Kubernetes-native metrics store
Visualization	Grafana	Dashboards, alerting	Rich visualization built on Prometheus data
IaC	Terraform	Environment provisioning	Declarative, version-controlled infrastructure

GitHub Actions is particularly suitable for AgriTwin because the codebase, pull requests, and CI/CD workflows all reside in a single GitHub repository, removing the need to manage a separate CI server or credentials for an external system such as Jenkins. The Software Engineering laboratory syllabus also covers CI/CD using Jenkins, and Jenkins remains a valid alternative for organizations that require self-hosted runners; however, no claim is made here that either tool has been executed against AgriTwin.

5. Automated Testing and Quality Assurance

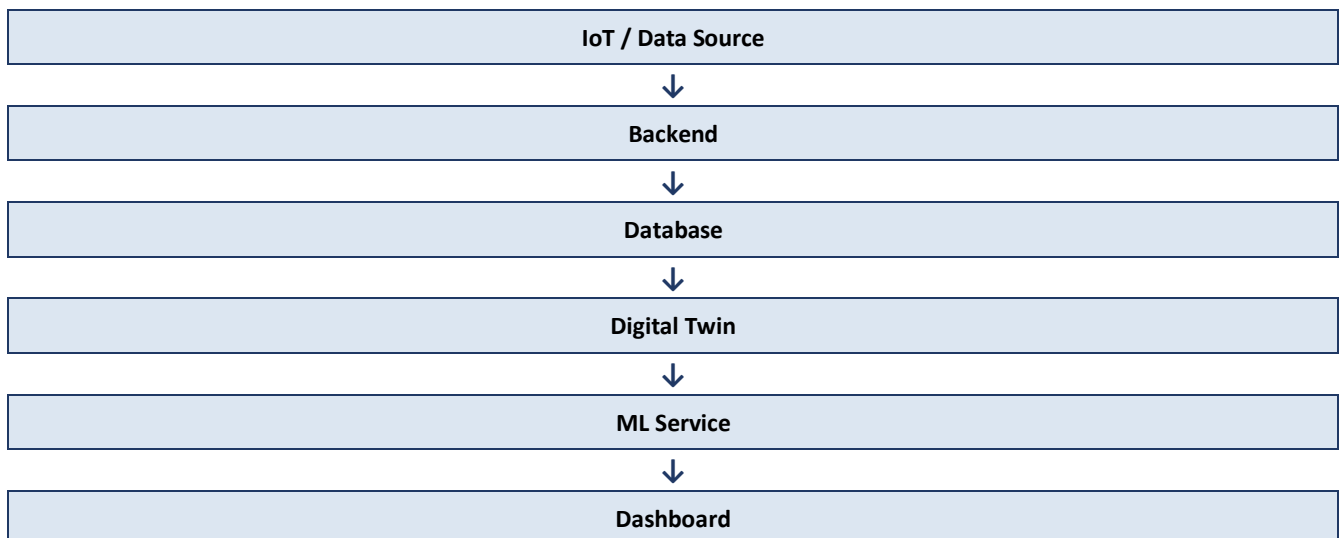
5.1 Unit Testing

- Authentication logic
- Sensor-data validation
- Crop data processing
- Recommendation logic

5.2 API Testing

- Authentication endpoints
- Sensor-data endpoints
- Crop-health prediction API
- Dashboard APIs
- Report APIs

5.3 Integration Testing



5.4 AI/ML Testing

- Input validation
- Preprocessing validation
- Model loading
- Prediction validation
- Model version verification
- Output schema validation
- Regression testing

5.5 Security Testing

- Dependency scanning
- Secret detection
- Container scanning

5.6 Code Quality

- Linting

- Static analysis
- Code coverage

Testing Pyramid

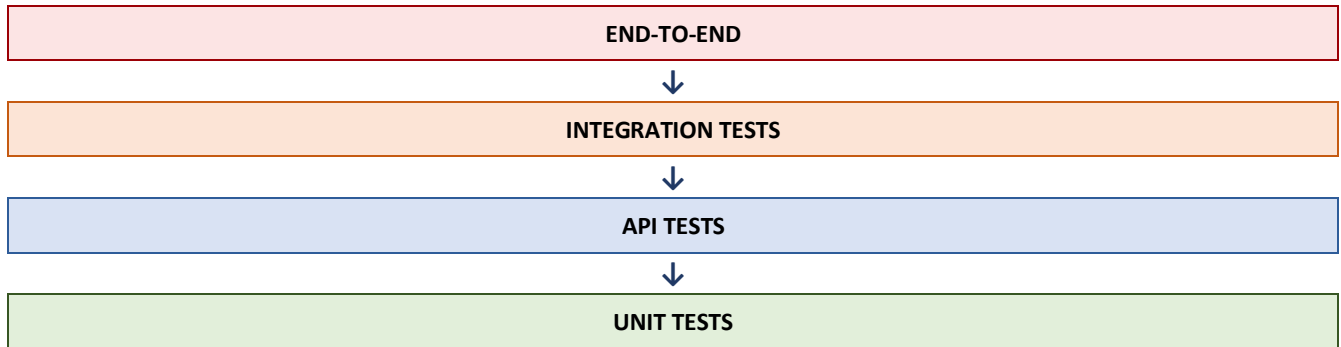


Figure 5.1: Testing pyramid — unit tests form the broad, fast base; end-to-end tests are fewest and slowest.

Most test coverage is concentrated in fast unit tests, with progressively fewer, slower tests at the API, integration, and end-to-end levels. This keeps pipeline feedback fast while still validating the system holistically before deployment.

Test Type	Tool	Purpose	Pipeline Action
Unit	Pytest	Validate isolated logic	Runs on every push/PR
API	Pytest + requests	Validate endpoint contracts	Runs after unit tests
Integration	Pytest	Validate cross-component flow	Runs before packaging
ML Validation	Pytest	Validate model I/O	Runs with integration suite
Coverage	Coverage.py	Measure test coverage	Reported after test run
Security	Trivy / GitHub	Find vulnerabilities	Runs before packaging and after build

6. Security and Quality Gates

AgriTwin follows DevSecOps principles, embedding security checks throughout the pipeline rather than treating them as a final, separate step.

6.1 Secure Source Code

- Branch protection on main
- Mandatory pull-request review
- Least-privilege access to repository and secrets

6.2 Secret Management

Credentials must never be committed to Git. Registry credentials, cloud credentials, database credentials, and API keys are stored using GitHub Secrets and injected into the pipeline only at runtime.

6.3 Dependency Security

Automated dependency vulnerability scanning identifies known CVEs in third-party Python packages before they reach production.

6.4 Container Security

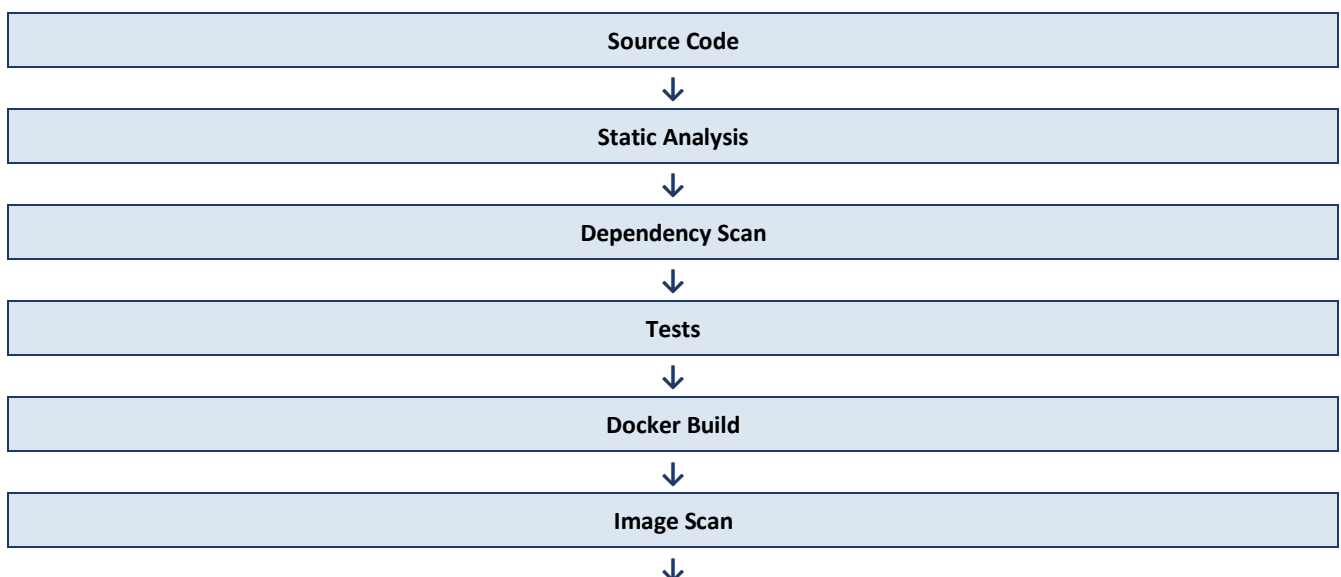
Trivy scans each built Docker image for OS and library vulnerabilities, blocking promotion when critical issues are found.

6.5 Quality Gates

The pipeline halts automatically when any of the following occur:

- Tests fail
- Lint fails
- A critical security vulnerability is detected
- Docker build fails
- Image scan fails
- Staging health check fails

Security Gate Flow



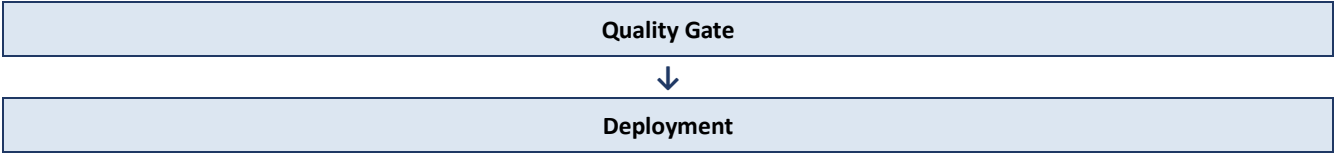


Figure 6.1: Security and quality gate sequence

7. Docker Containerization

Docker packages the AgriTwin application and its dependencies into a single, portable, immutable unit that runs consistently across development, staging, and production.

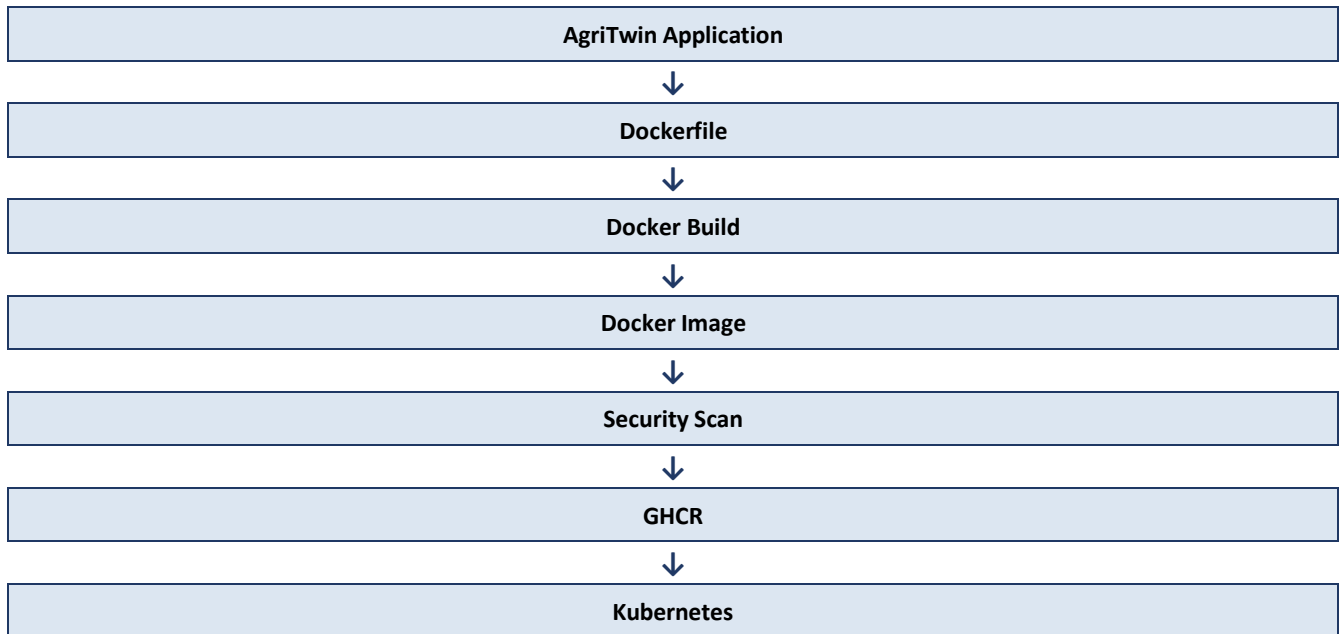


Figure 7.1: Docker packaging flow

Key Concepts

- Dockerfile – declarative build instructions for the application image.
- Image – an immutable, versioned snapshot of the application and its dependencies.
- Container – a running instance of an image.
- Registry – GHCR stores and versions built images.
- Immutable image – once built, an image is never modified, only replaced.
- Version tagging – each image is tagged for traceability.
- Reproducible environments – identical images run in every environment.

Illustrative Dockerfile

Illustrative example only — not claimed to have been executed.

```
FROM python:3.x-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Image Tagging and Rollback

Images are tagged with both a semantic version (for example agritwin:1.0.0) and the commit SHA (for example agritwin:<commit-sha>). Immutable, unique tags mean any previously built image can be redeployed exactly as it was, which is what makes fast, reliable rollback possible.

8. Deployment Strategy

AgriTwin is promoted through three progressively production-like environments.

Environments

- Development – developer-controlled environment for local iteration.
- Staging – production-like environment used for integration testing, smoke testing, and deployment verification.
- Production – the environment serving actual users.

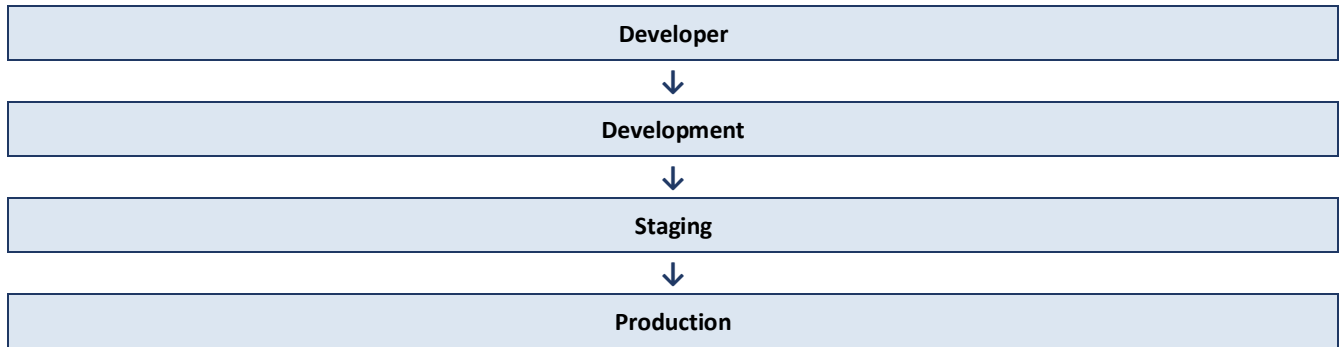


Figure 8.1: Environment promotion path

Deployment Strategy Comparison

Strategy	Advantage	Limitation	AgriTwin Suitability
Rolling Deployment	Zero downtime; gradual, resource-efficient rollout	Rollback is slightly slower than blue-green	Primary strategy — native Kubernetes support
Blue-Green Deployment	Instant switch-over and rollback	Requires double the infrastructure	Useful alternative when rapid rollback is required
A/B Deployment	Enables controlled feature experimentation	Added routing complexity; not needed for core releases	Reserved for future feature experiments

Rolling deployment is selected as AgriTwin's primary strategy because it is natively supported by Kubernetes, requires no additional infrastructure, and provides zero-downtime updates suited to a continuously monitored, scalable platform. Blue-green deployment is retained as an alternative for releases where near-instant rollback is critical, such as changes to the crop-health prediction service.

9. GitHub Actions CI/CD Workflow

Illustrative CI/CD configuration — not claimed to have been executed.

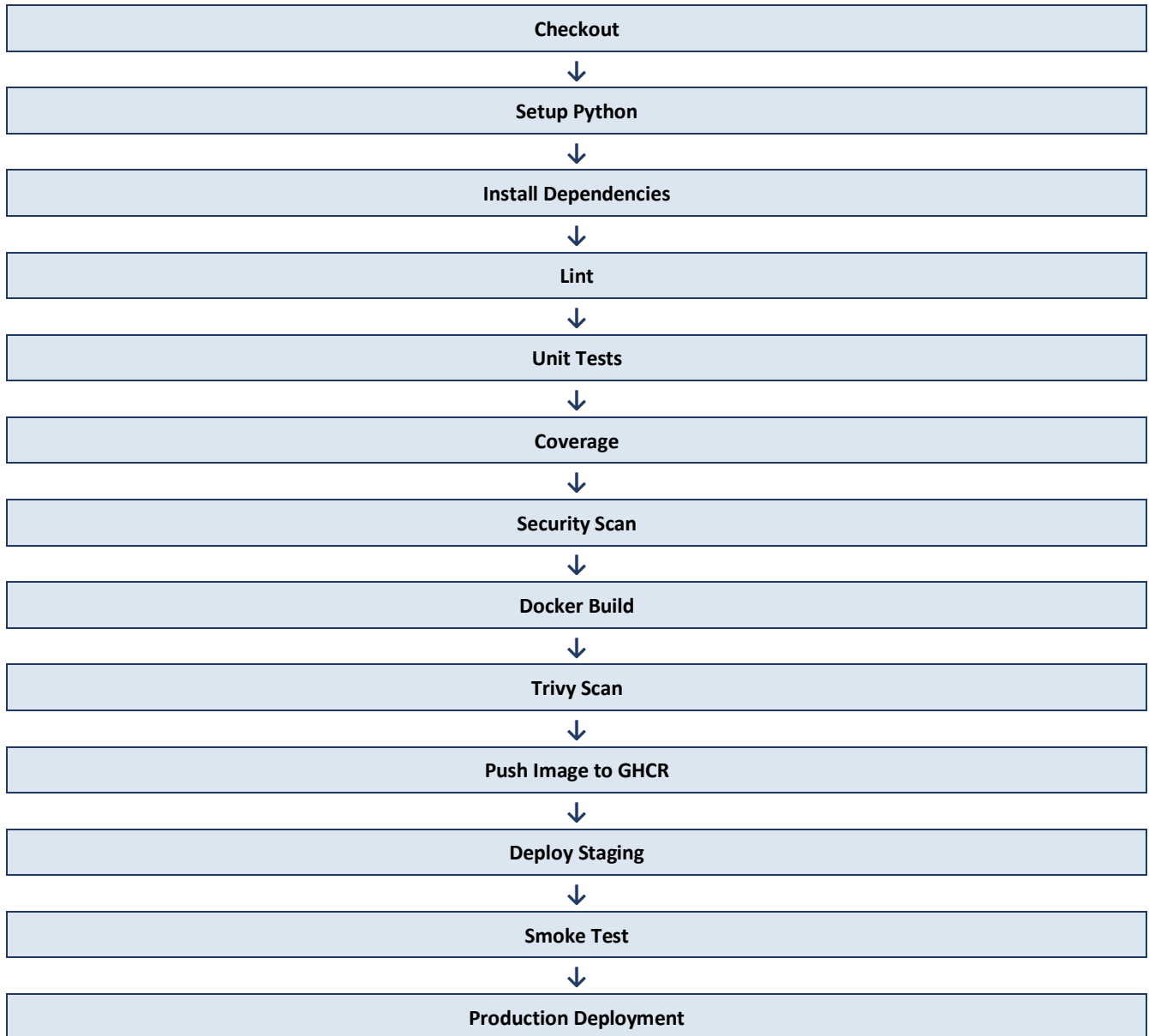


Figure 9.1: GitHub Actions job sequence

.github/workflows/ci-cd.yml (illustrative)

```

name: AgriTwin CI/CD
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r requirements.txt
      - run: ruff check .
      - run: pytest --cov=app
      - run: trivy fs .
  
```

```

build-image:
  needs: build-test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: docker build -t agritwin:${{ github.sha }} .
    - run: trivy image agritwin:${{ github.sha }}
    - run: docker push ghcr.io/org/agritwin:${{ github.sha }}

deploy-staging:
  needs: build-image
  runs-on: ubuntu-latest
  steps:
    - run: kubectl apply -f k8s/staging
    - run: pytest tests/smoke

deploy-production:
  needs: deploy-staging
  environment: production
  runs-on: ubuntu-latest
  steps:
    - run: kubectl apply -f k8s/production

```

Key elements: the workflow triggers on push and pull_request events; jobs run sequentially with explicit dependencies (needs); quality gates (lint, tests, scans) must pass before the image is built; GitHub Secrets supply registry and cluster credentials for Docker login and kubectl access; images are tagged with the commit SHA for traceability; and production deployment is gated behind a protected GitHub Environment requiring approval.

10. Kubernetes, Monitoring and Observability

10.1 Kubernetes Deployment

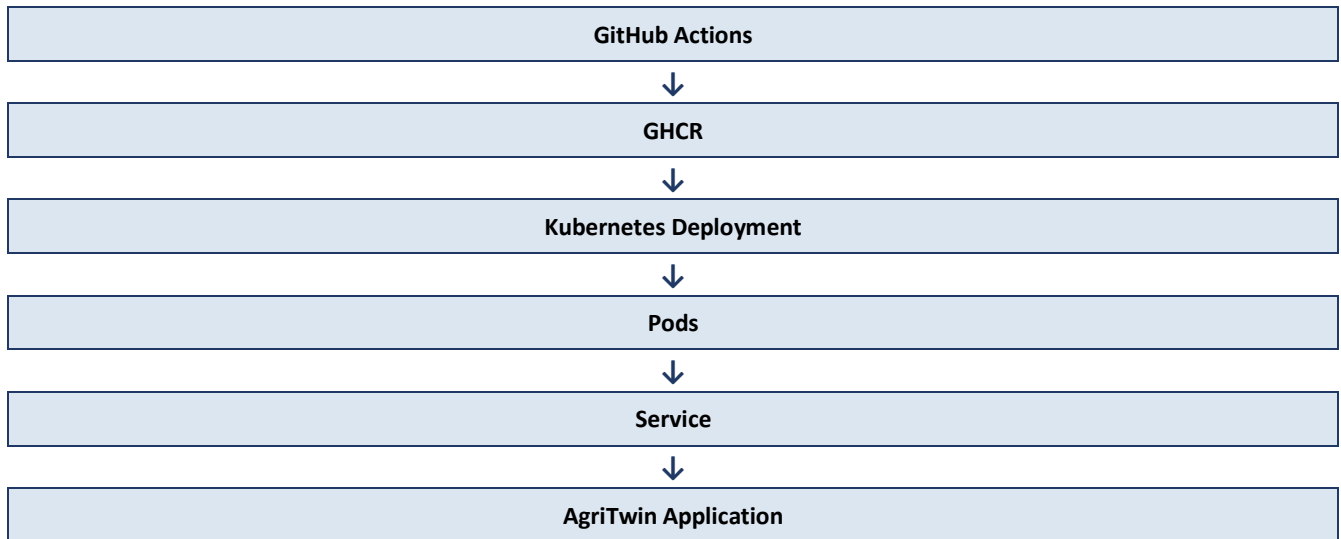


Figure 10.1: Kubernetes deployment flow

- Deployment – declares the desired state and manages pod replicas.
- Pods – smallest deployable units running the AgriTwin container.
- Service – stable network endpoint exposing the pods.
- Rolling update – replaces pods gradually with zero downtime.
- Readiness probe – confirms a pod can receive traffic.
- Liveness probe – confirms a running pod is still healthy.

10.2 Monitoring Architecture

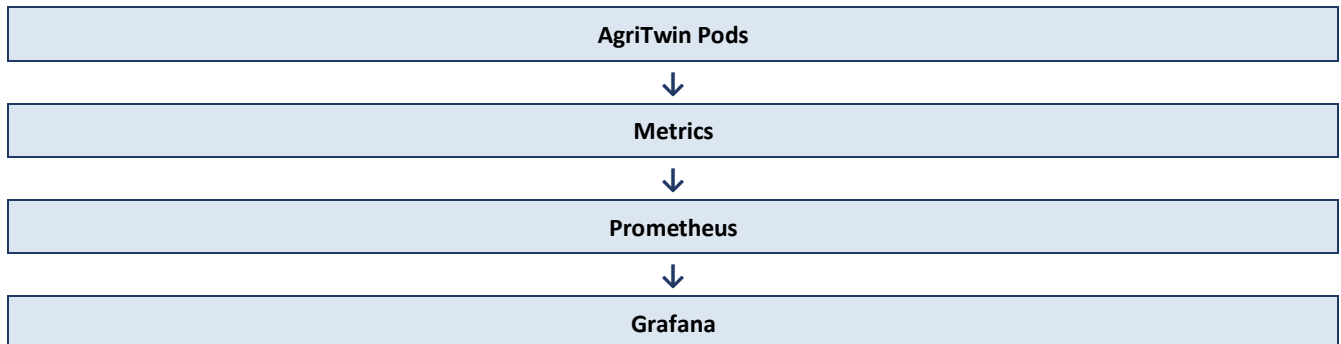


Figure 10.2: Metrics pipeline

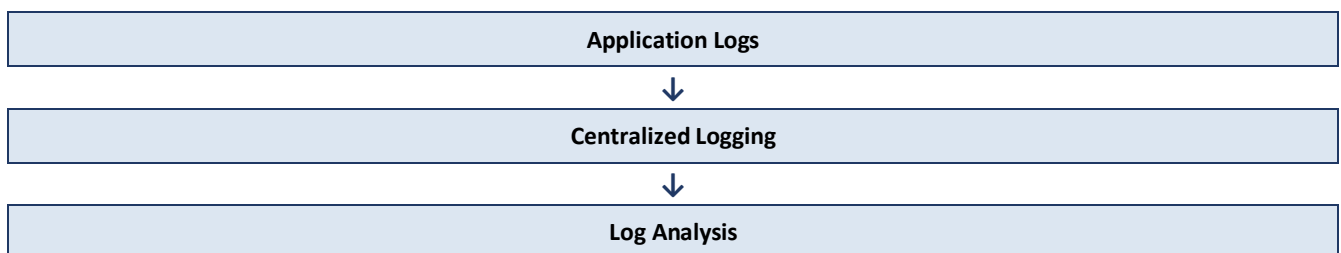


Figure 10.3: Logging pipeline

10.3 Key Monitoring Metrics

- CPU usage
- Memory usage

- Request count
- Response time
- Error rate
- Pod health
- Deployment status

11. Rollback and Failure Recovery

The pipeline anticipates several failure scenarios and defines a corresponding recovery response for each.

- Failed tests
- Failed Docker build
- Vulnerability detection
- Staging failure
- Production health-check failure
- Pod crash
- Application error

Rollback Flow

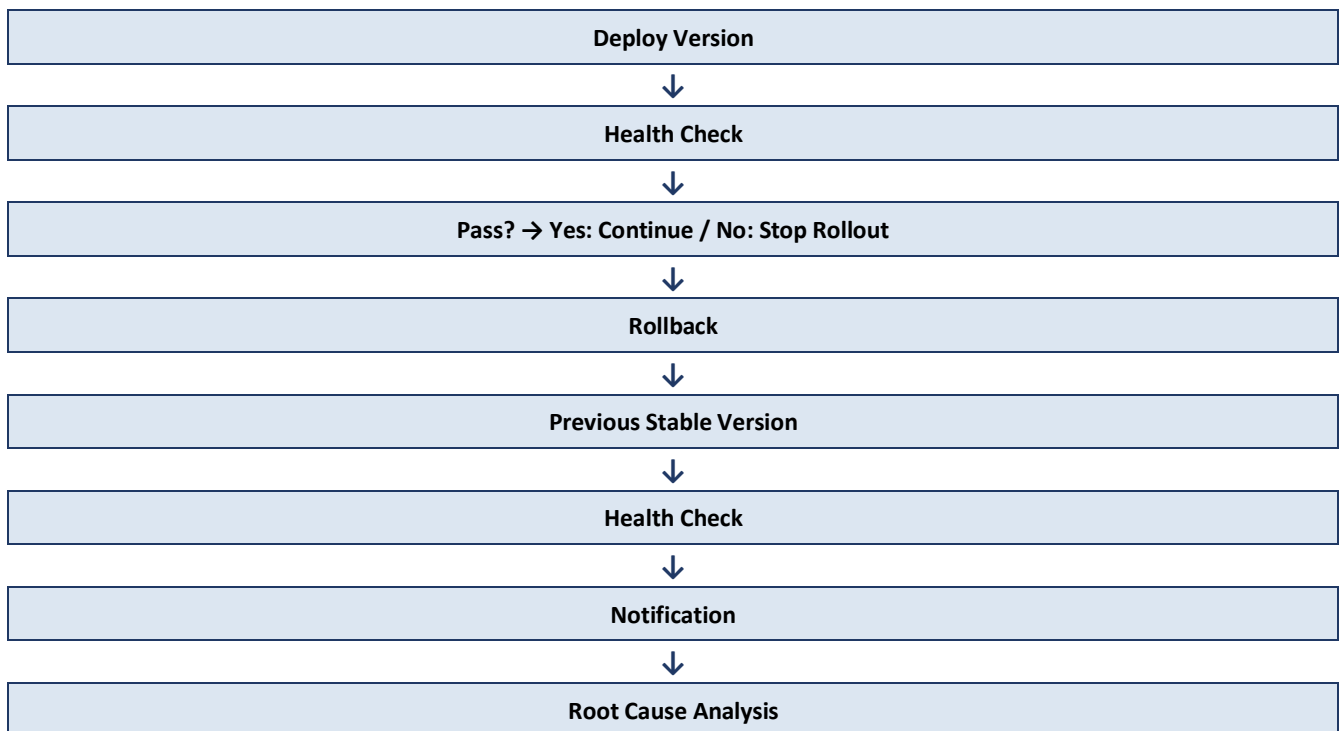


Figure 11.1: Rollback and recovery flow

Conceptual Rollback Example

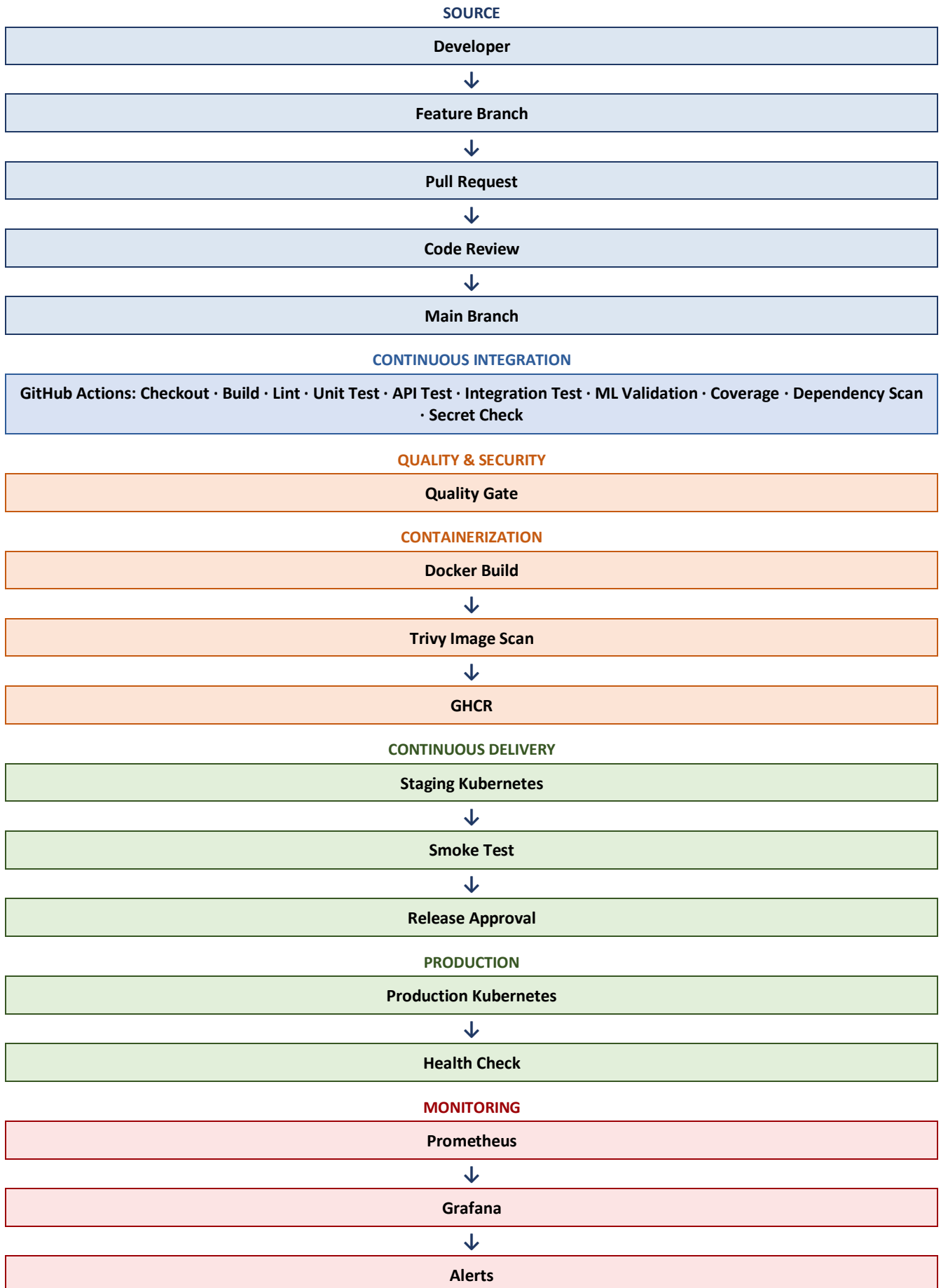
Illustrative example only — no rollback has been performed.

If the current release, tagged v1.2.0, fails its production health check, Kubernetes reverts traffic to the previous stable release, tagged v1.1.0: v1.2.0 → rollback → v1.1.0.

Recovery Mechanisms

- Automated rollback triggered by failed health checks
- Manual rollback for operator-initiated recovery
- Continuous health checks (readiness and liveness probes)
- Alerts routed to the responsible team
- Defined incident response procedure
- Root cause analysis following any rollback

12. Complete End-to-End CI/CD Pipeline





Rollback if Failure

Figure 12.1: Complete AgriTwin CI/CD Pipeline

13. Requirement Traceability and Justification

AgriTwin Requirement	CI/CD Mechanism	Tool	Expected Benefit
IoT data	Integration testing	Pytest	Reliable sensor data ingestion
Digital Twin	Backend testing	Pytest	Accurate field synchronization
AI/ML prediction	ML validation	Pytest	Consistent prediction pipeline
Crop-health prediction	Regression testing	Pytest	Stable model/API behaviour
Disease/pest detection	API + ML testing	Pytest	Early, reliable detection
Recommendations	API testing	Pytest	Correct recommendation output
Dashboards	Build/UI testing	CI pipeline	Reliable visualization
Predictive reports	API/integration testing	Pytest	Valid report generation
Scalability	Containerization	Docker / Kubernetes	Elastic scaling under load
Real-time monitoring	Health checks + metrics	Prometheus / Grafana	Rapid failure detection
Sustainability	Reliable, efficient releases	Full pipeline	Fewer wasted resources/incidents

Quality Attributes

Quality Attribute	Pipeline Mechanism
Reliability	Automated tests + health checks
Security	Dependency / image scanning
Scalability	Docker + Kubernetes
Maintainability	Git + code quality checks
Availability	Rolling deployment
Recoverability	Rollback
Observability	Prometheus + Grafana
Deployment Speed	GitHub Actions automation

Collectively, these mechanisms ensure that every AgriTwin release is tested, secured, observable, and recoverable, directly supporting the project's objectives of accurate crop-health prediction, efficient resource use, and a scalable, sustainable smart-agriculture platform.

14. Conclusion

This report has presented a complete CI/CD pipeline design for AgriTwin, a Digital Twin platform for precision farming and crop health prediction. The proposed architecture enables the project team to integrate changes safely, automatically test software at the unit, API, integration, and AI/ML levels, detect security issues in dependencies and container images early, and produce reproducible, versioned Docker images. Releases progress through a staging environment for verification before a controlled, monitored rollout to production. Prometheus and Grafana provide continuous visibility into application health, while an automated rollback mechanism allows any failed release to be reverted quickly to the last stable version. Together, these practices give AgriTwin a repeatable, secure, and reliable delivery process capable of supporting its goal of a scalable, data-driven smart-agriculture platform.

15. References

- GitHub Actions Documentation — <https://docs.github.com/actions>
- Docker Documentation — <https://docs.docker.com>
- Kubernetes Documentation — <https://kubernetes.io/docs>
- Prometheus Documentation — <https://prometheus.io/docs>
- Grafana Documentation — <https://grafana.com/docs>
- Trivy Documentation — <https://aquasecurity.github.io/trivy>
- Pytest Documentation — <https://docs.pytest.org>
- Terraform Documentation — <https://developer.hashicorp.com/terraform/docs>
- Ruff Documentation — <https://docs.astral.sh/ruff>

16. Evidence / Screenshot Checklist

All items below are to be captured during actual implementation. No execution is claimed in this report.

Evidence Item	Status
GitHub repository structure	Screenshot to be captured during actual implementation
GitHub Actions workflow run	Screenshot to be captured during actual implementation
Passing unit/API/integration tests	Screenshot to be captured during actual implementation
Coverage report	Screenshot to be captured during actual implementation
Dependency security scan result	Screenshot to be captured during actual implementation
Docker build log	Screenshot to be captured during actual implementation
Trivy container scan result	Screenshot to be captured during actual implementation
Image pushed to GHCR	Screenshot to be captured during actual implementation
Staging Kubernetes deployment	Screenshot to be captured during actual implementation
Production Kubernetes deployment	Screenshot to be captured during actual implementation

Evidence Item	Status
Prometheus metrics dashboard	Screenshot to be captured during actual implementation
Grafana dashboard	Screenshot to be captured during actual implementation
Rollback demonstration	Screenshot to be captured during actual implementation